Capstone — Group Task & Colab Code (Form → Sheet → Analyze → AI)

Your group's mission: build one complete data product. Create a **Google survey / application form** with an **image-document upload**, collect responses into a **Google Sheet**, then in **Google Colab** connect to that Sheet, **analyze and visualize** the data and write an interpretation, and finally use a **Teachable Machine** model to **identify the uploaded images** and **test the model's confidence**.

🚶 Fieldwork (group) — real respondents, real uploads

The real work: your group runs a real, small data product on **REAL people's submissions** — not your own recycled files. You collect, store, analyze, and let AI identify real uploaded documents, then **defend it**. *Totoong tao, totoong upload — hindi recycled na files.*

- Collect from at least **15 real respondents** outside your group who each **upload a real document/image** to your form.
- Train (or reuse) a Teachable Machine model on **REAL images your group gathered**.

PROOF (both required):

- The linked response **SHEET** showing the real submissions (timestamps spread over real time).
- PHOTOS + a **VIDEO** of your group collecting responses / people submitting — the field evidence.

Defense (the anti-LLM core): in your 3 minutes you must answer, live: **where did the data come from, who collected it, where could this pipeline fail, and what would you NOT trust it to decide.** A group that outsourced the thinking cannot survive these questions.

Shared rules: photo AND video both required (no proof = not accepted) · groups collect together but each student submits their own notebook/writeup · reduced/desk data only with teacher's prior permission (message first) · your raw response Sheet is a required deliverable.

How to work: open a new **Google Colab** notebook and add the code below cell by cell (one cell per

numbered block in Step 4), in order, then run from top to bottom.

Step 1 — Create the Google survey form (with image upload)

1. Go to **forms.google.com** → blank form. Title it (e.g. *Scholarship Application / Enrollment Form*).
2. Add a few questions: Name, Age or Grade, Course/Program, 1–2 multiple-choice questions.
3. Add a **File upload** question titled `Upload your document` → allow **1 file**, type **Image**.
4. **Responses** → green **Sheets** icon → **Create spreadsheet** (links a response Sheet).
5. Submit the form a few times, each time uploading an image (document / ID / form photo).

Important: a file-upload question makes responders **sign in with Google**, and uploads save to the **form owner's Drive**. The teammate who **owns the form** must run the Colab notebook (same Google account in the login popup) so it can read the images.

Step 2 — Get your Sheet ID + image column name

- Open the linked Sheet. The **Sheet ID** is the long code between `/d/` and `/edit` in the URL.
- Note the exact column header of the upload question (e.g. `Upload your document`).

Step 3 — Train your Teachable Machine model

1. Go to **teachablemachine.withgoogle.com** → **Image Project** → **Standard**.
2. Create your classes (e.g. *Valid / Not valid*) and add example images to each.
3. **Train Model**, then **Export Model** → **Tensorflow** → **Download my model** → you get a **.zip**.

Step 4 — The Colab code, in order

Run these in your notebook. Edit only the config in 4.4 (`SHEET_ID` and `IMAGE_COL`).

4.1 — Install the tools

```
!pip install gspread -q
```

4.2 — Import the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os, io, re
import gspread
from google.colab import auth
from google.auth import default
from googleapiclient.discovery import build
from googleapiclient.http import MediaIoBaseDownload
from IPython.display import display
print("Libraries loaded.")
```

4.3 — Sign in to Google (Sheets + Drive)

```
# ============= CONNECT TO GOOGLE (Sheet + Drive) =============
# A popup asks you to choose your Google account (the one that OWNS the form/sheet) and Allow.
auth.authenticate_user()
creds, _ = default()
gc = gspread.authorize(creds)
drive_service = build('drive', 'v3')
print("Connected to Google Sheets + Drive.")
```

4.4 — Load your form responses (set SHEET_ID + IMAGE_COL)

```
# ============= LOAD YOUR FORM RESPONSES =============
# Paste your linked response Sheet's ID, and the exact column title of the IMAGE UPLOAD question.
SHEET_ID  = 'PASTE_YOUR_SHEET_ID_HERE'
IMAGE_COL = 'Upload your document'   # change to your form's file-upload question title

ws = gc.open_by_key(SHEET_ID).get_worksheet(0)
df = pd.DataFrame(ws.get_all_records())
print(f"Loaded {len(df)} responses with {df.shape[1]} columns.")
print("Columns:", list(df.columns))
display(df.head())
```

4.5 — Explore & describe the data

```
# ============= EXPLORE & DESCRIBE THE DATA =============
print("Data types:")
print(df.dtypes)

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
if num_cols:
    print("\nNumeric summary:")
    display(df[num_cols].describe().round(2))

print("\nMissing values per column:")
miss = df.isnull().sum()
print(miss[miss > 0] if miss.sum() else "  None")
```

4.6 — Visualize the responses

```
# ============= VISUALIZE THE FORM DATA =============
# Bar charts for short text/choice answers; histograms for numbers.
skip = {IMAGE_COL, 'Timestamp', 'Email Address', 'Email address'}
cat_cols = [c for c in df.columns
            if c not in skip and df[c].dtype == object and 1 < df[c].nunique() <= 15]
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()

plots = cat_cols[:3] + num_cols[:3]
if plots:
    n = len(plots); cols = 3; rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(6*cols, 4.2*rows))
    axes = np.array(axes).flatten()
    for i, c in enumerate(plots):
        if c in num_cols:
            axes[i].hist(df[c].dropna(), bins=10, color='steelblue', edgecolor='black', alpha=0.8)
        else:
            vc = df[c].value_counts()
            axes[i].bar(vc.index.astype(str), vc.values, color='teal', alpha=0.85)
            axes[i].tick_params(axis='x', rotation=45)
        axes[i].set_title(str(c)[:40], fontweight='bold')
    for j in range(len(plots), len(axes)):
        axes[j].axis('off')
    plt.tight_layout(); plt.show()
else:
    print("No chartable columns detected yet — collect a few more responses.")

print("\nINTERPRETATION (write 2-3 sentences): what stands out in the responses above?")
```

Then write your interpretation (2–3 sentences): what stands out about your applicants / enrollees?

4.7 — Upload your Teachable Machine model .zip (auto-arranged)

```
# ============= UPLOAD YOUR TEACHABLE MACHINE MODEL (.zip) =============
# Export Model -> Tensorflow -> "Download my model" gives a .zip. Upload it here; it auto-arranges.
import zipfile, glob, shutil
from PIL import Image
from google.colab import files

print("Choose your Teachable Machine model .zip ...")
uploaded = files.upload()
zip_names = [n for n in uploaded if n.lower().endswith('.zip')]
if not zip_names:
    raise SystemExit("No .zip found. Re-run and select your model zip.")

MODEL_ROOT = '/content/model'
if os.path.exists(MODEL_ROOT):
    shutil.rmtree(MODEL_ROOT)
os.makedirs(MODEL_ROOT, exist_ok=True)
with zipfile.ZipFile(zip_names[0]) as z:
    z.extractall(MODEL_ROOT)

savedmodel = glob.glob(f'{MODEL_ROOT}/**/saved_model.pb', recursive=True)
keras_files = glob.glob(f'{MODEL_ROOT}/**/*.h5', recursive=True)
label_files = glob.glob(f'{MODEL_ROOT}/**/labels.txt', recursive=True)
labels_path = label_files[0] if label_files else None
print("Found -> SavedModel:", bool(savedmodel), "| Keras:", bool(keras_files), "| labels:",
bool(labels_path))
```

4.8 — Load the model + prediction function

```
# ============= LOAD THE MODEL + PREDICTION FUNCTION =============
import tensorflow as tf

if savedmodel:
    model = tf.saved_model.load(os.path.dirname(savedmodel[0]))
    predict_fn = model.signatures['serving_default']; MODE = 'savedmodel'
elif keras_files:
    model = tf.keras.models.load_model(keras_files[0], compile=False)
    predict_fn = None; MODE = 'keras'
else:
    raise SystemExit("No model found in the zip.")

class_names = []
if labels_path:
    with open(labels_path) as f:
        for line in f:
            t = line.strip()
            if t:
                p = t.split(' ', 1)
                class_names.append(p[1] if len(p) == 2 and p[0].isdigit() else t)
print(f"Model loaded ({MODE}). Classes:", class_names)

def predict_image(path):
    img = Image.open(path)
    if img.mode != 'RGB':
        img = img.convert('RGB')
    arr = np.asarray(img.resize((224, 224))).astype(np.float32)
    x = (arr / 255.0)[None, ...] if MODE == 'savedmodel' else ((arr / 127.5) - 1.0)[None, ...]
    if MODE == 'savedmodel':
        out = predict_fn(tf.constant(x)); probs = out[list(out.keys())[0]].numpy()[0]
    else:
        probs = model.predict(x, verbose=0)[0]
    idx = int(np.argmax(probs))
    label = class_names[idx] if idx < len(class_names) else f'Class {idx}'
    return label, float(probs[idx])
```

4.9 — Identify each uploaded document

```
# ============= CLASSIFY EACH UPLOADED DOCUMENT + CONFIDENCE =============
def extract_drive_id(url):
    m = re.search(r'[-\w]{25,}', str(url))
    return m.group(0) if m else None

def download_drive_file(file_id, dest):
    req = drive_service.files().get_media(fileId=file_id)
    fh = io.FileIO(dest, 'wb')
    dl = MediaIoBaseDownload(fh, req); done = False
    while not done:
        _, done = dl.next_chunk()
    fh.close(); return dest

os.makedirs('/content/form_images', exist_ok=True)
preds, confs = [], []
for i, row in df.iterrows():
    fid = extract_drive_id(row.get(IMAGE_COL, ''))
    if not fid:
        preds.append(None); confs.append(None); continue
    try:
        dest = download_drive_file(fid, f'/content/form_images/row{i}.jpg')
        label, conf = predict_image(dest)
        preds.append(label); confs.append(conf)
    except Exception as e:
        print(f"Row {i}: {e}"); preds.append(None); confs.append(None)

df['AI_Prediction'] = preds
df['AI_Confidence'] = confs
done = df.dropna(subset=['AI_Confidence'])
print(f"Classified {len(done)} of {len(df)} uploaded documents.")
display(df[[IMAGE_COL, 'AI_Prediction', 'AI_Confidence']].head(20))
```

4.10 — Test the model's confidence

```
# ============= TEST THE MODEL'S CONFIDENCE =============
if len(done) == 0:
    print("No images classified yet — check SHEET_ID, IMAGE_COL, and that responses have uploads.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    axes[0].hist(done['AI_Confidence'], bins=10, color='gold', edgecolor='black')
    axes[0].axvline(done['AI_Confidence'].mean(), color='red', linestyle='--',
                    label=f"Mean {done['AI_Confidence'].mean():.1%}")
    axes[0].axvline(0.8, color='orange', linestyle=':', label='80% review line')
    axes[0].set_title('Model Confidence Distribution'); axes[0].set_xlabel('Confidence');
axes[0].legend()

    by = done.groupby('AI_Prediction')['AI_Confidence'].mean().sort_values()
    axes[1].barh(by.index, by.values, color='teal')
    axes[1].set_xlim(0, 1); axes[1].set_title('Average Confidence by Predicted Class')
    plt.tight_layout(); plt.show()

    print("CONFIDENCE REPORT")
    print(f"  Average confidence: {done['AI_Confidence'].mean():.1%}")
    print(f"  High (>=90%): {(done['AI_Confidence'] >= 0.9).sum()}  |  "
          f"Low (<80%, review): {(done['AI_Confidence'] < 0.8).sum()}  of {len(done)}")
    print("  Predictions per class:")
    for cls, n in done['AI_Prediction'].value_counts().items():
        print(f"    {cls}: {n}")
    print("\nINTERPRETATION: Is the model confident? Which uploads need a human to review (<80%)?")
```

3-hour timetable

Time	What happens
0:00–0:15	Brief, groups, pick form type + model classes
0:15–1:00	Build the form (with image upload) + collect a few responses
1:00–1:45	Connect Colab to the Sheet → analyze + visualize + interpret
1:45–2:30	Train/reuse the Teachable Machine model → classify the uploads → test confidence
2:30–3:00	Polish (findings + 1 recommendation) + dry-run the 3-min story
Hour 4	Presentations & closing

Save time: reuse a Teachable Machine model from Week 2 instead of training a fresh one.

Deliverables (tick to pass)

- ☐ Google Form with image upload + linked Sheet
- ☐ Notebook connected to the Sheet
- ☐ Charts + written interpretation
- ☐ Each uploaded image identified by the model
- ☐ Confidence tested (average + items <80%)
- ☐ 3-minute story

Scoring rubric (20 points)

Criteria	Points
Form + data — working form with image upload, responses in a Sheet	4
Analysis & visualization — notebook connected, clear charts	4
Interpretation — sensible reading of the data	4
AI identification — model classifies the uploaded images	4
Confidence + presentation — confidence tested; clear 3-min story ending in a decision	4
Total	**20**

📌 Proof & Submit — required to complete the capstone

1. Your notebook / Colab link (connect Sheet → analyze → visualize → classify → confidence).
2. Your linked response **Sheet** (the raw real submissions).
3. Your PROOF: the PHOTO(s) **and** the VIDEO of your group collecting responses.
4. Your written interpretation + the 3-minute defense notes.

No fieldwork proof = the capstone is NOT accepted, even if the code runs. *Kung hindi kayo makakakuha ng totoong respondents, mag-message muna sa teacher BEFORE the activity.*

Under the DPA / privacy notice: only collect data people agree to give; for photos/videos of people, ask their okay first; don't collect sensitive personal info you don't need.